

2.1 Project Planning

Document 2.1 — Project Planning

Project: Hospital Appointment Management System **Platform:** ServiceNow (Scoped Application) **Application Scope:** x_2065601_hospit_0 **Phase:** Project Planning

1. Project Scope Statement

In Scope

- Patient registration and profile management
- Doctor registration and profile management (specialization, qualification, consultation fee, shift, max patients per day)
- Doctor-managed availability slot creation, viewing, and blocking
- Real-time slot browsing and booking by patients
- Atomic slot booking transaction to prevent double-booking
- Appointment lifecycle management: Booked, Checked-In, Completed, Cancelled, No-Show
- Waitlist registration for fully booked doctors/dates
- Automated waitlist promotion when a slot is freed via cancellation or no-show
- Front Desk/Admin manual booking for walk-in patients
- Role-based Service Portal with dedicated tabs for Patient, Doctor, and Front Desk/Admin
- Backend data verification across all core tables (Patient, Doctor, Doctor Availability Slot, Appointment, Waitlist)
- Basic reporting/list views for administrative oversight

Out of Scope

- Billing and payment processing
- Prescription management and clinical notes
- Electronic Health Record (EHR) integration
- SMS/email reminder integrations (third-party gateway dependency)
- Telemedicine/video consultation
- Insurance verification and claims processing
- Native mobile application (web-based Service Portal only)

2. User Stories

#	User Story
1	As a patient , I want to register my profile with basic details , so that I can book appointments under my own record .
2	As a patient , I want to view real-time available slots for a doctor , so that I can choose a time that works for me without calling the clinic .
3	As a patient , I want to book an appointment instantly , so that I have a confirmed, guaranteed slot without risk of double-booking .
4	As a patient , I want to view my appointment history , so that I can track past and upcoming visits in one place .
5	As a patient , I want to cancel an appointment I no longer need , so that the slot can be freed up for another patient .
6	As a patient , I want to join a waitlist when my preferred slot is full , so that I am automatically considered if that slot becomes available .
7	As a doctor , I want to register my profile with my

	specialization and consultation details , so that patients can find and book me correctly .
8	As a doctor , I want to add and manage my own availability slots , so that patients can only book times I have actually made available .
9	As a doctor , I want to view my daily patient queue , so that I know exactly who is booked, checked-in, or has completed their consultation .
10	As a doctor , I want to mark a patient as checked-in, completed, or no-show , so that the appointment record accurately reflects what happened .
11	As a front-desk/admin staff member , I want to manually book an appointment for a walk-in patient , so that patients without portal access are still accommodated .
12	As a front-desk/admin staff member , I want to view all doctors' schedules in one place , so that I can coordinate bookings without cross-checking multiple registers .
13	As a system , I want to automatically promote the next waitlisted patient when a slot is cancelled , so that no available slot goes unused .

3. Sprint / Phase Breakdown

Phase	Name	Description
Phase 0	Requirements & Design	Define scope, personas, tables, and portal structure
Phase 1	Data Model	Build core tables: Patient, Doctor, Doctor Availability Slot, Appointment, Waitlist
Phase 2	Slot Generation	Implement doctor-side slot creation and availability logic
Phase 3	Booking Logic	Implement atomic patient booking transaction and conflict prevention
Phase 4	Lifecycle & Automation	Implement appointment state transitions and waitlist auto-promotion
Phase 5	Patient Portal	Build Patient Registration and Patient Booking & History tabs
Phase 6	Doctor/Front Desk UI	Build Doctor Registration, Doctor Queue & Slots, and Front Desk/Admin tabs
Phase 7	Reporting	Build basic administrative list views and oversight reporting
Phase 8	Testing	Functional, ACL/role, and concurrency testing across all flows
Phase 9	Documentation	Compile capstone documentation, test reports, and execution guide

4. Milestones Table

Phase	Milestone	Estimated Effort/Duration
Phase 0	Requirements finalized, tables and scope agreed	2 days
Phase 1	Core tables (Patient, Doctor, Slot, Appointment, Waitlist) created with fields	3 days

Phase 2	Doctor availability slot generation functional	2 days
Phase 3	Atomic booking transaction implemented and verified	3 days
Phase 4	Appointment lifecycle states and waitlist auto-promotion working	4 days
Phase 5	Patient portal tabs (Registration, Booking & History) complete	4 days
Phase 6	Doctor and Front Desk/Admin portal tabs complete	4 days
Phase 7	Reporting/list views available for admin oversight	2 days
Phase 8	Full test cycle (functional, ACL, concurrency) executed	3 days
Phase 9	Final documentation package compiled	3 days

5. Risks Identified and Mitigation

Risk	Description	Mitigation Approach
Double-booking race conditions	Two patients could attempt to book the same slot simultaneously, resulting in a conflict	Implement the booking action as a single atomic transaction that checks and flips slot status and creates the appointment record together, rather than as separate steps
Field-mapping errors between UI and backend tables	Widget form fields could be misaligned with actual table field names, causing silent save failures	Explicitly map every portal form field to its corresponding backend field name during development and verify with test bookings visible in list views
Waitlist promotion not triggering reliably	Cancelling or marking a no-show might not consistently trigger promotion of the next waitlisted patient	Use a dedicated, testable business rule/event triggered specifically on slot status change to Available, verified through repeated test cases
Role-based access errors (ACL misconfiguration)	A patient, doctor, or front-desk user could gain access to data outside their role's intended scope	Define and test ACLs per role against each core table before UI development is considered complete, using dedicated role-based test cases
Incomplete or inconsistent portal-to-table sync	Portal actions (e.g., cancellation) might not always reflect correctly in the backend table state	Cross-verify every portal workflow against the underlying ServiceNow table in list view as part of the standard testing checklist
Scope creep toward out-of-scope features	Ideas such as SMS reminders or telemedicine could expand the build beyond capstone timeline	Maintain the documented in-scope/out-of-scope boundary from this planning document and defer such features to the "Future Enhancements" section of the final capstone document

End of Document 2.1